How to use "Bhex's Over-Named Extension for Mod-Sized Datapacks" ("BONE:MSD")

BONE:MSD is an extension/api datapack for creating mod-like experiences with minecraft Datapacks. The goal is to remove as many steps of the process as possible to create a smooth, data-driven experience for adding custom content to your datapack.

Custom Blocks in BONE:MSD are based on Barriers with custom components which get replaced when the Player places them. Using the "Custom Data" component, this pack is able to automatically link your block with its placeable item without the need to manually add your block to a list or check for tagged Item Frames. This also ensures your block placement rules are consistent with standard Minecraft Blocks.
Custom Blocks can be enabled with Redstone Functionality.

Custom items in BONE:MSD are based on vanilla items, sometimes with their right-click functionality removed. These items are similar to traditional Custom Items in datapacks.

Blocks and Items are added to the creative inventory using Painting Variations. BONE:MSD automatically converts these Painting Variations into Custom Blocks and Items once they enter your inventory.

Included with the download of BONE:MSD is an example datapack which should give you a good idea of where to start. There are many ways to use this pack, so a single tutorial would not be able to cover the full scope and use-cases of this pack.

KNOWN ISSUES:

- When placing a doubleslab, the texture changes for 1 tick before reverting to its fullblock texture.
- Blocks which use Entities can be interfered with by pistons, so it is advisable to use immovable blocks or blocks that break when pushed by a piston as these base blocks.

***Waterlogged Doubleslab Only:**

- When breaking a Custom Block, water is visible for 1 tick.
- Custom Blocks act like waterlogged blocks, because they are. Which means lava and pistons are weird about them and they can’t be blown up.
- Custom Blocks can be turned into Double Slabs by right-clicking them with an empty bucket.
- Custom Blocks are affected by Sponges.

Datapack Format

In order for the Macros in this extension datapack to work, you must build your datapack with a specific file structure. The file structure is visualized below in ASCII.

```
─── Datapack Folder/
    ├── pack.png
    ├── pack.mcmeta
    └── data/
        └── example_pack/
            ├── loot_table/
            │   ├── blocks/
            │   │   ├── example_block/
            │   │   │   └── give.json
            │   │   └── second_example_block/
            │   │       └── give.json
            │   └── items/
            │       └── example_item/
            │           └── give.json
            ├── function/
            │   └── blocks/
            │       ├── example_block/
            │       │   └── place.mcfunction
            │       └── second_example_block/
            │           └── place.mcfunction
            └── paintings/
                ├── blocks/
                │   ├── example_block.json
                │   └── second_example_block.json
                └── items/
                    └── example_item.json
```

The "Function" folder format is optional, but useful especially if starting from scratch.

Adding Custom Blocks

Loot Tables

To allow your block to be placed using a custom item, create a Loot Table which drops a Barrier. Use the Item Name and Item Model components to make it look like your Custom Block, but the base item must be a Barrier.
The following must be inserted into the "custom data" component to enable your item to become a placeable block:

```
"bone_cb": {
    "pack": "example_pack",
        "block_name": "example_block",
        "use_place_function": true|false,
        "place_function_location": "example_pack:blocks/example_block/place",
        "place_information": {
            "base_block": "spawner",
            "block_entity": {
                "id": "item_display",
                "entity_data": "{Tags:[test]}"
                "loot_table": "example_pack:blocks/example_block/break"
        }
    }
}
```

The "pack" field requires the namespace of your datapack folder. For Vanilla Minecraft, this is "minecraft". This refers to the namespace folder in the "data" folder.

The "block_name" field is the name of the folder where your block's individual function files are located.

The "use_place_function" boolean is required and defines whether the pack will look for a Place Function for your block or use its built-in API to place a block using the "place_information" field.

The "place_command_location" field is where you define a custom path for your block's "place" function. If no path is defined, then the default path is used. The default location that the Pack looks for Place functions is constructed using the data from these fields. The final path should look something like:

example_pack:blocks/example_block/place
This field is ignored if "use_place_function" is set to "false".

The "place_information" field is an api to make blocks placeable without creating a "place" function yourself. This works for simple blocks, but more complex blocks with multiple entities, models, or blocks may require a dedicated "place" function. This field is ignored if "use_place_function" is set to "true".

The "base_block" field is what the base block of your block will be. It is possible to define blockstates and NBT in this field.

The "block_entity" field defines the entity that will be summoned and used for the block. This Entity will automatically be tagged with "bone_msd_block" and the contents from the "block_name" field for tracking with commands. If this field is left empty or "use_place_function" is set to "true", no entity will be summoned.

The "id" field defines the type of entity that will be summoned when the block is placed. This is usually a Marker or Display Entity.

The "entity_data" field defines the NBT data of the entity that is summoned.

The "loot_table" field defines the loot table that is used when the block is broken. If the field is empty, then the block will use its "give" loot table as its drop. This is included in the "block_entity" field due to this field requiring an entity with the block in order to work.

Redstone Functionality

In order to make a Custom Block interact with redstone, you will need to assign an entity to the custom block, and give the entity the "redstone_powered" tag.

To make use of this Entity detecting redstone power, use score selectors.
If you want a function to only run once when the block is powered, check the entity for an "impulse" score of 1.
If you want a function to run continuously when the block is powered, check the entity for an "power" score of 1 or higher.
To filter specific power levels, you can check for specific values of the "power" score.

The function files for redstone functionality are in the functions/utils/redstone folder.

Double-Slab Blocks

Vanilla double-slabs have been, mostly, removed from the game. When a double-slab is placed by the player, it will turn into a full-block of the same type. The only exception is Smooth Stone Slabs, due to their texture.

This turns double-slabs into un-used blockstates, similar to waterlogged double-slabs or cracked pots. This allows you to use double-slabs as custom blocks in your datapack with no drawbacks. These blocks also use no entities by default and work exactly as expected from normal blocks. However, the loot-tables will need modifying to integrate custom block drops.

Block Breaking

BONE custom blocks that use an entity have a built-in "break" function. This detects its base block being broken and replaces the dropped item with either the loot table defined in the Custom Block's data, or the item from the "give" loot table for the block.

Adding Custom Items

Custom Items can be added using Components like with most packs.

Disable "place" function

You can disable most items' right-click function(except for bows, buckets, and food) by giving it the "minecraft:can_break" component and including the "#bone_msd:item_engine/noplace" tag. This disables the right-click functions of items which can be "placed". IE, Spawn Eggs, Item Frames, Paintings, and most blocks.
This component makes the Item Tooltip unusably large, so you must hide the "minecraft:can_break" tooltip using the "Tooltip Display" field.

Holding a Custom Item with this component in your off-hand may interfere with gameplay in edge cases, however these are rare and simply a method of waterproofing the Extension.

Use In Recipes

Since there is no Component Filtering for Crafting Ingredients, you must use a work-around solution to use your Custom Items in recipes without cross-play with Vanilla Survival materials. It is recommended to use Operator Utility items as your base items. These blocks cannot be obtained or used by normal players in Survival Mode and are not used in any Vanilla recipes, making them a perfect surrogate item for Custom Crafting ingredients. Should you need more items that can be used in recipes, you can use Spawn Eggs as base items, provided those Spawn Eggs are not available through normal gameplay. These items will appear in their original form in the Recipe Book if their models and translation strings are not overwritten.

Creative Inventory

You add Custom Block and Items to the Creative Inventory using Paintings. Create a Painting with a Variation and Asset path that matches the path to the corresponding "give" loot table. For example:
example_pack:blocks/example_block.json
example_pack:items/example_item.json

Use a Resource Pack to change the model of Paintings based on their Variations.
This allows the Extension to automatically replace these paintings with Custom Block and Items from their loot table, assuming the corresponding tables exist. Paintings which do not share a path structure with a loot table will be ignored.

A custom font and translate strings are used to cover up the Vanilla Item Text.

Is it recommended to add these paintings to the "painting_variant/placeable" tag. This allows the blocks to be moved to the "functional blocks" tab, allowing Creative players to access these blocks without the Operator Utilities tab.
If you do not want to include custom paintings in your pack, you can double-up on Vanilla painting textures.

The tooltip for Custom Blocks and Items in the inventory is large. There is no way around that.

Raycast Utility

The pack features a very basic built-in raycast utility. This raycast has a step size of 1 cm and a maximum reach of 7 blocks. It stops when it collides with any block. It is not hitbox-accurate.

To use the Raycast requires two commands.

function bone_msd:utils/raycast/trigger

This command triggers the Raycast. An entity gets summoned at the "hit" location. To execute at the "hit" location, use the following command:

execute as @e[tag=raycast_hit] at @s run {command}

This command MUST be run in the same tick and before any other raycast functions are called, as only one of these "hit" markers can exist in the world at once, so it is best to put these two commands back-to-back in a function. If a persistent Marker is needed, then simply summon a unique Marker at the "hit" location.